Teorías de Autómatas

y Lenguajes Formales

Colección manuales uex - 55

(E.E.E.S.)

55

Elena
Jurado Málaga

Tema 4

Autómatas Finitos

Contenido

4.1. Introducción 39
4.2. Definición de Autómata Finito Determinista 40
4.3. Representación de Autómatas 40
4.4. Los AFD como reconocedores de lenguajes 43
4.5. Minimización de un AFD 43
4.6. Autómatas Finitos No Deterministas(AFND) 47
4.7. Lenguaje aceptado por un AFND 50
4.8. Simulación de un AFD y AFND 51
4.9. Paso de un AFND a AFD 52
4.10. Relación entre AF, gr. y exp. reg. 55
4.11. Límites para los leng. regulares 65
4.12. Problemas 70

4.1. Introducción

Aunque no se puede considerar como una definición correcta de autómata, está muy extendida una idea que confunde el concepto de autómata con el de robot. Por lo tanto, se considera erróneamente que un autómata es una máquina que imita funciones típicas de los seres vivos, sobre todo relacionadas con el movimiento, pudiendo incluso ejecutar ciertas órdenes. En realidad el concepto de autómata es mucho más genérico, ya que podemos considerarlo como un dispositivo que procesa cadenas de símbolos que recibe como entrada, cambiando de estado y produciendo una salida que, en algunos casos, puede estar formada por otra cadena de símbolos.

La teoría de autómatas se ocupa de clasificar y estudiar de modo sistemático diferentes tipos de máquinas abstractas que llevan a cabo un procesamiento secuencial de la información. Dentro del conjunto de las máquinas abstractas que estudiaremos en esta asignatura, los Autómatas Finitos constituyen el grupo de máquinas más sencillas y que, por tanto, tienen un menor poder funcional.

El estudio de los autómatas finitos se utiliza para modelar el comportamiento de dispositivos mecánicos y también de sistemas naturales. Concretamente, permite estudiar procesos que dependen de una *historia*, es decir, sistemas cuyo comportamiento actual depende del pasado. También se aplican en el procesamiento del lenguaje natural, pero en el contexto de esta asignatura su principal aplicación será el reconocimiento de lenguajes regulares (de tipo 3).

4.2. Definición de Autómata Finito Determinista

Los Autómatas Finitos son máquinas teóricas que van cambiando de estado dependiendo de la entrada que reciban. La salida de estos Autómatas está limitada a dos valores: *aceptado* y *no aceptado*, que pueden indicar si la cadena que se ha recibido como entrada es o no válida. Generalmente utilizaremos los Autómatas Finitos para reconocer lenguajes regulares, es decir, una palabra se considerará válida sólo si pertenece a un determinado lenguaje.

Formalmente, un Autómata Finito Determinista (AFD) se define como una tupla

$$AFD = (\Sigma, Q, f, q_0, F), \text{ donde}$$

Σ es el alfabeto de entrada

Q es el conjunto finito y no vacío de los estados del Autómata

f es la función de transición que indica en qué situaciones el Autómata pasa de un estado a otro, se define $f : Q \times \Sigma \longrightarrow Q$

$q_0 \in Q$ es el estado inicial

$F \subset Q$ es el conjunto de estados finales de aceptación ($F \neq \emptyset$)

4.3. Representación de Autómatas

Existen dos formas de representar un AFD, mediante **tablas de transición** o mediante **diagramas de transición**. Introduciremos estas dos representaciones con un ejemplo.

Sea el siguiente AFD: $\Sigma = \{a, b\}$ $\quad Q = \{p, q, r\}$ $\quad q_0 = p$ $\quad F = \{q\}$

donde f se define de la siguiente forma:

$f(p, a) = q$	$f(p, b) = r$
$f(q, a) = q$	$f(q, b) = r$
$f(r, a) = r$	$f(r, b) = r$

Tabla de transición El AFD se representaría mediante la siguiente tabla que representa los valores de la función de transición.

		a	b
$\rightarrow$	p	q	r
$*$	q	q	r
	r	r	r

La flecha indica que p es el estado inicial, y el asterisco indica que q es un estado final de aceptación (en general, pueden aparecer muchos asteriscos aunque sólo puede aparecer una flecha ya que sólo hay un estado inicial).

Diagrama de transición La figura 4.1 representa de forma gráfica las transi-

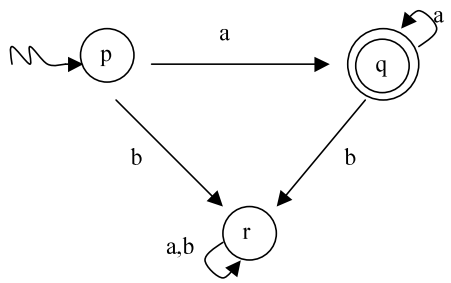


Figura 4.1: Ejemplo de AFD

ciones del autómata. Los estados finales de aceptación se identifican por estar encerrados en un doble circulo. El estado inicial se destaca con una flecha *arrugada*.

Al analizar el autómata del ejemplo es evidente que sólo considera como cadenas aceptadas aquellas que están formadas solamente por **a**'s. Cualquier cadena que contenga una **b** hará que el autómata acabe en el estado $\boxed{\text{r}}$, que es un estado *muerto*. Diremos que un estado está *muerto* si no es un estado final de aceptación y no parte de él ninguna transición hacia otro estado. Es evidente que si durante el análisis de una cadena se llega a un estado *muerto*, como ya no es posible *salir* de dicho estado, la cadena no será aceptada por el autómata.

Autómatas Incompletos A menudo nos encontramos con autómatas para los que no están definidas todas las transiciones. Las situaciones que no están definidas deben ser consideradas como *situaciones de error*, es decir, si una cadena hace llegar al autómata hasta una situación no definida, consideraremos que la cadena no ha sido reconocida por dicho autómata.

Si deseamos completar un autómata (no es imprescindible) bastará con añadir un estado *muerto* que reciba todas las transiciones que le faltan al autómata incompleto.

En la figura 4.2 podemos ver un ejemplo de esta situación. El autómata de la izquierda está incompleto, pero podemos completarlo trasformándolo en el de la derecha, al que hemos añadido el estado **r**, que es un estado *muerto*.

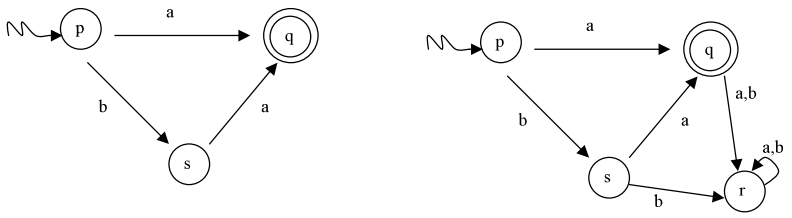


Figura 4.2: Ejemplo de AF incompleto y completo

Estados accesibles y autómatas conexos

Definición 4.1 (Autómata Conexo)
Un autómata es conexo si todos sus estados son accesibles desde el estado inicial.

Definición 4.2 (Parte conexa de un autómata)
Si un autómata no es conexo, se llama parte conexa del autómata al conjunto de estados accesibles desde el estado inicial.

Ejemplo 4.1 El autómata representado en la figura 4.3 no es conexo y su parte conexa es la formada por los estados **p** y **q** y por las transiciones que hay entre ellos.

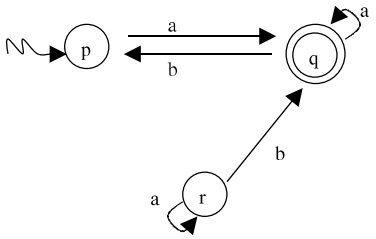


Figura 4.3: AF no conexo

4.4. Los AFD como reconocedores de lenguajes

Como se ha visto en secciones anteriores, la función de transición f ha sido definida de manera que depende de un único símbolo del alfabeto de entrada. A continuación, se ampliará esta definición de forma que dicha función pueda *actuar* sobre cadenas de símbolos, es decir, la función indicará a qué estado pasa el autómata ante la llegada de una cadena de símbolos (y no solamente de un único símbolo). Por tanto, consideraremos que $f : Q \times \Sigma^* \longrightarrow Q$. Para conseguir esta ampliación, f se redefine de forma recursiva:

- $f(q, \lambda) = q \qquad \forall q \in Q$
- $f(q, ax) = f(f(q, a), x) \qquad \forall a \in \Sigma,\ x \in \Sigma^*,\ q \in Q$

Ejemplo 4.2 Considerando el autómata que aparece en la figura 4.1 la función de transición extendida devolvería los siguientes valores:

$f(p, a) = q$	$f(p, aa) = q$
$f(p, ab) = r$	$f(p, aabbb) = r$
$f(p, baba) = r$	$f(r, abb) = r$

Con esta nueva definición de la función de transición es posible definir formalmente cuál es el lenguaje aceptado por un AFD.

Definición 4.3 (Leng. aceptado por un AFD)
El lenguaje que acepta un AFD es el conjunto de palabras definidas sobre Σ que hacen que el autómata llegue a un estado final de aceptación

$$L = \{x \in \Sigma^* / \ f(q_0, x) \in F\}$$

4.5. Minimización de un AFD

En ocasiones nos encontramos con autómatas que tienen algunos estados equivalentes, en estos casos esos estados se pueden agrupar de manera que se consigue un autómata, equivalente al primero, pero con un menor número de estados. Se dice que el autómata ha sido minimizado. Un concepto diferente al de minimizar un autómata es el de *simplificar* un autómata que consiste en eliminar estados muertos o inaccesibles.

A continuación se presenta un algoritmo para minimizar un AFD. El objetivo principal de este algoritmo consiste en agrupar estados equivalentes. Consideraremos que dos estados son equivalentes cuando las transiciones que parten de ellos, para cada uno de los símbolos del alfabeto, llevan al mismo estado o a estados que también son

equivalentes entre sí. Todos los estados que sean equivalentes entre sí se fundirán en un único estado en el autómata resultante. Para conseguir este objetivo se construirá una partición de Q, que se irá refinando paulatinamente, de manera que finalmente cada elemento de la partición agrupará estados equivalentes. Inicialmente, se construye una partición de Q formada por dos únicos elementos: los estados de aceptación y los que no lo son. Dicha partición se irá refinando todo lo posible, separando en diferentes elementos a los estados que no son equivalentes. Recordemos que una partición de Q consiste en dividir Q en varios subconjuntos $\{G_i\}_{1\leq i\leq n}$ de tal forma que:

$$G_i \cap G_j = \emptyset \ \forall i \neq j \quad y \quad \bigcup_{1\leq i\leq n} G_i = Q$$

Algoritmo 4.1 Minimización de un AFD

Input: $AFD\ A = (\Sigma, Q, f, q_0, F)$
Output: $AFD\ A' = (\Sigma, Q', f', q_0', F')$
Begin
Partición $= \{G_1, G_2\}$ donde $G_1 = F$ y $G_2 = Q \setminus F$
Auxiliar $= \emptyset$
while Auxiliar $\neq$ Partición **do**
Auxiliar = Partición
$\forall G_i \in$ Partición y $\forall a \in \Sigma$ separar en diferentes grupos a los estados
s y $t \in G_i$ siempre que $f(s,a) \in G_j, f(t,a) \in G_k$ siendo $j \neq k$
end while
Cada elemento G_i de Partición se convierte en un estado de Q', las transiciones serán las mismas que define f
End

Ejemplo 4.3 Se minimizará el Autómata representado en la figura 4.4.

Inicialmente, Partición $= \{G_1, G_2\}$ $\quad G_1 = \{E\}$ $\quad G_2 = \{A, B, C, D\}$

Evidentemente no es posible refinar el grupo G_1. Pero hay que comprobar cómo se comportan los estados de G_2 con los símbolos a y b.

G_2		símbolo a	G_2		símbolo b
A	$\rightarrow$	$B \in G_2$	A	$\rightarrow$	$C \in G_2$
B	$\rightarrow$	$B \in G_2$	B	$\rightarrow$	$D \in G_2$
C	$\rightarrow$	$B \in G_2$	C	$\rightarrow$	$C \in G_2$
$\boldsymbol{D}$	$\rightarrow$	$\boldsymbol{B \in G_2}$	$\boldsymbol{D}$	$\rightarrow$	$\boldsymbol{E \in G_1}$

Analizando el comportamiento de los cuatro estados con el símbolo b, es evidente que D no es equivalente a los otros tres estados. Por tanto, es necesario dividir G_2 en dos nuevos elementos.

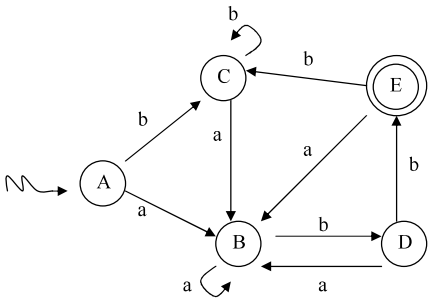


Figura 4.4: Autómata a minimizar

Partición $= \{G_1, G_2', G_3\}$ $\quad G_1 = \{E\}$ $\quad G_2' = \{A, B, C\}$ $\quad G_3 = \{D\}$
Veamos como se comporta G_2' con los símbolos a y b.

G_2'		símbolo a	G_2'		símbolo b
A	$\rightarrow$	$B \in G_2'$	A	$\rightarrow$	$C \in G_2'$
$\boldsymbol{B}$	$\rightarrow$	$\boldsymbol{B \in G_2'}$	$\boldsymbol{B}$	$\rightarrow$	$\boldsymbol{D \in G_3}$
C	$\rightarrow$	$B \in G_2'$	C	$\rightarrow$	$C \in G_2'$

De nuevo es evidente que el estado B no es equivalente a los otros dos. Es necesario dividir G_2'. Ahora, Partición $= \{G_1, G_2'', G_3, G_4\}$ donde
$G_1 = \{E\}$ $\quad G_2'' = \{A, C\}$ $\quad G_3 = \{D\}$ $\quad G_4 = \{B\}$
Volvemos a comprobar el comportamiento de G_2'', el único grupo que es posible refinar.

G_2''		símbolo a	G_2''		símbolo b
A	$\rightarrow$	$B \in G_4$	A	$\rightarrow$	$C \in G_2''$
C	$\rightarrow$	$B \in G_4$	C	$\rightarrow$	$C \in G_2''$

No es posible refinar más la partición. Es evidente que A y C son equivalentes y, por tanto, deben formar parte del mismo grupo. A continuación aparece el nuevo autómata (figura 4.5), equivalente al anterior, que tiene un estado menos, debido al agrupamiento entre A y C(representado por G_2'').

Definición 4.4 (AFD equivalentes)
Dos AFD son equivalentes si reconocen el mismo lenguaje. Es evidente que para que esto ocurra deben estar definidos sobre el mismo alfabeto.

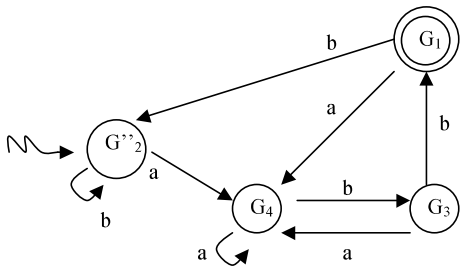


Figura 4.5: Autómata minimizado

A veces es fácil comprobar de manera intuitiva si dos AFD son equivalentes, pero esto no siempre ocurre. Un método para comprobar si dos AFD son equivalentes consiste en *unirlos* de manera que formen un único AFD que, por supuesto, no es conexo. Formalmente, la unión de los dos autómatas se llevaría a cabo así:

Sean $A_1 = \{\Sigma, Q_1, f_1, q_{01}, F_1\}$ $\quad A_2 = \{\Sigma, Q_2, f_2, q_{02}, F_2\}$

El autómata resultante tras la *unión* sería

$A = A_1 + A_2 = \{\Sigma, Q_1 \cup Q_2, f, q_0, F_1 \cup F_2\}$ donde

$$f(q,a) = \begin{cases} f_1(q,a) & si\ q \in Q_1 \\ f_2(q,a) & si\ q \in Q_2 \end{cases}$$

El nuevo estado inicial q_0 puede ser tanto q_{01} como q_{02}.

Una vez construido el autómata *unión*, éste se minimiza. Si al concluir el proceso de minimización, los dos estados iniciales q_{01} y q_{02} forman parte del mismo elemento de la partición, los autómatas originales son equivalentes y el AFD que hemos obtenido es el autómata mínimo equivalente a ambos.

Ejemplo 4.4 Dados los autómatas de la figura 4.6, aplicaremos el método anterior para decidir si son equivalentes o no.

Como hemos visto inicialmente, Partición $= \{G_1, G_2\}$

donde $\quad G_1 = \{q, r, w\} \quad G_2 = \{p, s, v, u\}$

Hay que comprobar cómo se comportan los elementos de G_1 y G_2 con los símbolos a y b.

G_1		símbolo a	G_1		símbolo b
q	$\rightarrow$	$r \in G_1$	q	$\rightarrow$	$p \in G_2$
r	$\rightarrow$	$q \in G_1$	r	$\rightarrow$	$p \in G_2$
w	$\rightarrow$	$w \in G_1$	w	$\rightarrow$	$v \in G_2$

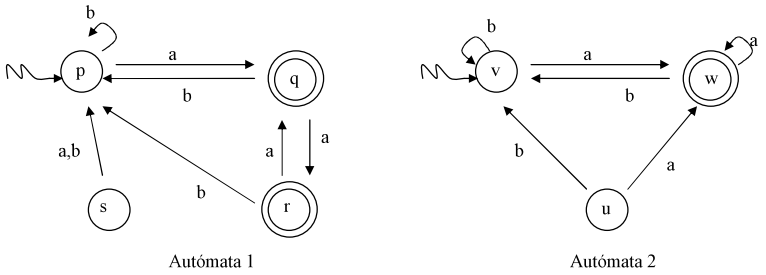


Figura 4.6: ¿Autómatas equivalentes?

G_2		símbolo a	G_2		símbolo b
p	$\rightarrow$	$q \in G_1$	p	$\rightarrow$	$p \in G_2$
$\boldsymbol{s}$	$\rightarrow$	$\boldsymbol{p \in G_2}$	$\boldsymbol{s}$	$\rightarrow$	$\boldsymbol{p \in G_2}$
v	$\rightarrow$	$w \in G_1$	v	$\rightarrow$	$v \in G_2$
u	$\rightarrow$	$w \in G_1$	u	$\rightarrow$	$v \in G_2$

En principio, parece que los estados del elemento G_1 son equivalentes, sin embargo, analizando el comportamiento de los estados del elemento G_2 con el símbolo a, es evidente que s no es equivalente a los otros tres estados. Por tanto, es necesario dividir G_2 en dos subgrupos.

Partición $= \{G_1, G_2', G_3\}$ $\quad G_1 = \{q, r, w\}$ $\quad G_2' = \{p, v, u\}$ $\quad G_3 = \{s\}$

Veamos cómo se comporta G_2' con los símbolos a y b.

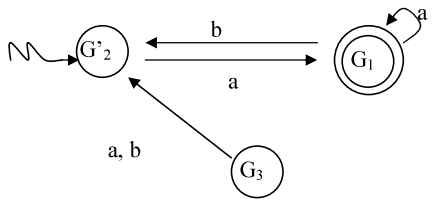
G_2'		símbolo a	G_2'		símbolo b
p	$\rightarrow$	$q \in G_1$	p	$\rightarrow$	$p \in G_2'$
v	$\rightarrow$	$w \in G_1$	v	$\rightarrow$	$v \in G_2'$
u	$\rightarrow$	$w \in G_1$	u	$\rightarrow$	$v \in G_2'$

Es evidente que todos los estados de G_2' son equivalentes. Por tanto, no es posible refinar más la partición. Podemos determinar que los Autómatas 1 y 2 son equivalentes ya que los estados iniciales p y v son equivalentes. Además, el autómata que se muestra en la figura 4.7 es equivalente a ambos y es mínimo.

Como el estado G_3 es inaccesible se puede eliminar. Así el autómata quedaría como muestra la figura 4.8.

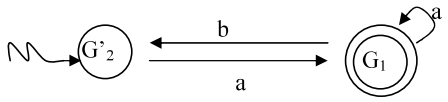
4.6. Autómatas Finitos No Deterministas(AFND)

En los autómatas deterministas sabemos exactamente cuál es la transición que debemos llevar a cabo ante una determinada situación. Sin embargo, en los no deterministas podemos encontrarnos con varias opciones e, incluso, con λ-transiciones



$G_1 = \{q, r, w\}$ $G'_2 = \{p, u, v\}$ $G_3 = \{s\}$

Figura 4.7: Autómata mínimo



$G_1 = \{q, r, w\}$ $G'_2 = \{p, u, v\}$

Figura 4.8: Autómata simplificado

que se realizan sin considerar el correspondiente símbolo de la cadena de entrada.

Para tener en cuenta estas consideraciones, los AFND se definen como una tupla: $AFND = (\Sigma, Q, f, q_0, F, T)$, $\quad f : Q \times \Sigma \longrightarrow 2^Q$ donde

- 2^Q es el conjunto formado por los subconjuntos de Q, incluyendo a $\emptyset$
- T es una relación binaria definida sobre Q que indica las λ-transiciones del autómata (si $pTq \Rightarrow$ existe una λ-transición desde p hasta q)

El resto de los símbolos tiene el mismo significado que en la definición de AFD.

Ejemplo 4.5 Representación de un AFND utilizando un diagrama de transiciones: ver figura 4.9

Descripción del mismo autómata mediante una tabla de transiciones:

	a	b	λ
$\rightarrow *p$	$\{q\}$	$\emptyset$	$\emptyset$
q	$\{p,s\}$	$\{r,p\}$	$\{s\}$
r	$\emptyset$	$\{s,p\}$	$\{r,s\}$
$*s$	$\emptyset$	$\emptyset$	$\{r\}$

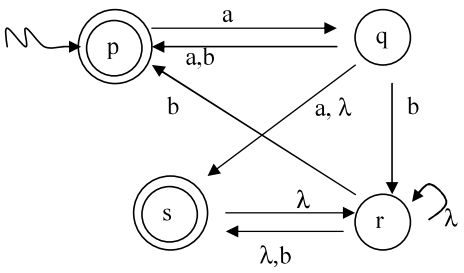


Figura 4.9: A.F. no determinista

Es evidente que un AFD no es más que un caso particular de AFND, es decir, AFD $\subset$ AFND. En realidad, un AFD es un AFND que cumple

$$T = Id \quad y \quad |f(q,a)| = 1 \quad \forall q \in Q,\ \forall a \in \Sigma$$

Es útil conocer el cierre transitivo de la relación T, que se denota T^*. Si pT^*q, entonces q es accesible desde p utilizando exclusivamente λ-transiciones.

Para calcular T^* podemos utilizar una matriz booleana (B_T) que permita representar a T y calcular después todas las potencias de dicha matriz (llega un momento en que las potencias se repiten y no es necesario calcular más). Así $B_{T^2} = (B_T)^2$ representa las parejas de estados que están conectadas por dos λ-transiciones, B_{T^3} las que están conectadas por tres, y así sucesivamente. Por lo tanto $Id + B_T + B_{T^2} + B_{T^3} + \ldots$ es una matriz booleana que representa la relación que deseamos calcular T^*. En nuestro ejemplo:

$$B_T = \begin{array}{c} p \\ q \\ r \\ s \end{array} \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} (B_T)^2 = B_{T^2} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

$$B_{T^3} = B_{T^4} = \ldots = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix} B_{T^*} = B_{Id} + B_T + B_{T^2} + B_{T^3} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

Definición 4.5 (λ-clausura)
Sea $q \in Q$, se llama λ-clausura(q) al conjunto de estados de Q que son accesibles desde q mediante λ-transiciones. Por lo tanto,

$$\lambda - \text{clausura}(q) = \{p \in Q / qT^* p\}$$

Esta definición se puede ampliar a conjuntos de estados de manera natural.

Si $R \subset Q$, entonces $\lambda - \text{clausura}(R) = \bigcup_{q \in R} \lambda - \text{clausura}(q)$

Aunque para calcular la λ-clausura de un estado se pueden utilizar las matrices booleanas que acabamos de ver, a continuación se describe un algoritmo que también nos permite calcular la λ-clausura.

Algoritmo 4.2 Cálculo de la λ-clausura(q)

Output: Clausura $\subset Q$
Begin
Clausura = $\{q\}$
Auxiliar = $\emptyset$
while Auxiliar $\neq$ Clausura **do**
Auxiliar = Clausura
Clausura = Clausura $\cup$ $\{s \in Q / \exists$ λ-transición desde p hasta s, siendo $p \in$ *Auxiliar*$\}$
end while
λ-clausura(q) = Clausura
End

4.7. Lenguaje aceptado por un AFND

El lenguaje aceptado por un AFND es el conjunto de todas las cadenas de símbolos terminales que pueden hacer que el AFND llegue a un estado final de aceptación. Para llegar a una definición formal de este lenguaje ampliaremos la definición de la función de transición con objeto de que acepte cadenas de caracteres. Es decir, si la función de transición de un AFND de define así: $f : Q * \Sigma \to 2^Q$. Definiremos una función de transición *ampliada*, de la siguiente forma $f' : Q * \Sigma^* \to 2^Q$, donde

- $f'(q, \lambda) = \lambda - \text{clausura}(q)$

- $f'(q, ax) = \{p \in f'(r, x)/r \in f'(q, a)\} =$
  $= \{p \in Q/\ \exists r \in f'(q, a) \text{ y } p \in f'(r, x)\}$ siendo $a \in \Sigma,\ x \in \Sigma^*$

Una vez ampliada la definición de la función de transición, el lenguaje aceptado por el AFND es: $L(AFND) = \{x \in \Sigma^*/f'(q_0, x) \cap F \neq \emptyset\}$

4.8. Simulación de un AFD y AFND

En esta sección veremos sendos algoritmos que nos permitirán simular el comportamiento de un AFD y de un AFND. Por tanto, permitirán determinar si una cadena pertenece o no al lenguaje que reconoce el autómata.

Algoritmo 4.3 Simulación de un AFD

Input: $x \in \Sigma^*$
Begin
  c es el primer carácter de x
  $q = q_0$
  while c $\neq$ Fin **do**
    $q = f(q, c)$
    c es el siguiente carácter de x
  end while
  if $q \in F$ **then**
    la palabra x ha sido reconocida por el AFD
  else
    la palabra x no ha sido reconocida por el AFD
  end if
End

Para detallar el algoritmo de simulación de un AFND, supondremos que tenemos implementadas las siguientes funciones:

- $f(R, a) = \cup_{q \in R} f(q, a)$, siendo $R \subset Q$ y $a \in \Sigma$

- $\lambda - \text{clausura}(R)$, siendo $R \subset Q$

La diferencia fundamental entre ambos algoritmos está en el significado de q y S. En el algoritmo 4.3, q representa el estado que el autómata tiene en cada instante. Sin embargo, en el algoritmo 4.4, q es sustituido por S que representa al conjunto de los estados en los que *puede* estar el autómata.

Algoritmo 4.4 Simulación de un AFND

Input: $x \in \Sigma^*$
Begin
c es el primer carácter de x
$S = \lambda - \text{clausura}(q_0)$
while c $\neq$ Fin **do**
$S = \lambda - \text{clausura}(f(S, c))$
c es el siguiente carácter de x
end while
if $S \cap F \neq \emptyset$ **then**
la palabra x ha sido reconocida por el AFND
else
la palabra x no ha sido reconocida por el AFND
end if
End

4.9. Paso de un AFND a AFD

Los AFND y los AFD tienen el mismo poder computacional (esto no ocurre en otros niveles de la jerarquía de los autómatas), es decir, pueden resolver los mismos problemas. Por lo tanto, dado un AFND siempre es posible encontrar un AFD que sea equivalente a él. En esta sección estudiaremos un método para resolver este problema. En primer lugar explicaremos de manera genérica el paso de un autómata a otro, para después ilustrar con un ejemplo este mecanismo de trasformación.

Partimos de un AFND $= (\Sigma, Q, f, q_0, F, T)$ y queremos construir un AFD $= (\Sigma, Q', f', q'_0, F')$ que sea equivalente, donde:

1. $Q' = 2^Q$
2. $q'_0 = \lambda - \text{clausura}(q_0)$
3. $F' = \{C \subset Q / C \cap F \neq \emptyset\}$
4. $f'(C, a) = \{C' \subset Q / C' = \bigcup_{q \in C} \lambda - \text{clausura}(f(q, a))\}$, siendo $C \subset Q$

El autómata que se obtiene por este método no tiene porqué ser mínimo y podría llegar a tener, como máximo, $2^{|Q|}$ estados.

Ejemplo 4.6 Calcularemos el AFD equivalente al AFND que se muestra en la figura 4.10. Comenzamos calculando el estado inicial q_0 que es la λ-clausura del estado inicial del AFND. $q_0 = \lambda$-clausura(A) = {A,C}

A continuación hay que calcular la función de transición para el estado q_0

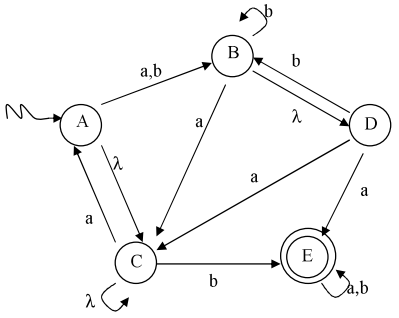


Figura 4.10: Ejemplo

$$
\begin{array}{rcl}
f(q_0,a) & = & \lambda\text{-clausura}(f(A,a)\cup f(C,a)) = \lambda\text{-clausura}(B,A) = \{A,B,C,D\} = q_1 \\
f(q_0,b) & = & \lambda\text{-clausura}(f(A,b)\cup f(C,b)) = \lambda\text{-clausura}(B,E) = \{B,D,E\} = q_2
\end{array}
$$

Es necesario seguir calculando la función de transición para los nuevos estados que van apareciendo.

$$
\begin{array}{rcl}
f(q_1,a) & = & \lambda\text{-clausura}(f(A,a)\cup f(B,a)\cup f(C,a)\cup f(D,a)) \\
 & = & \lambda\text{-clausura}(B,C,A,E) = \{A,B,C,D,E\} = q_3 \\
f(q_1,b) & = & \lambda\text{-clausura}(f(A,b)\cup f(B,b)\cup f(C,b)\cup f(D,b)) \\
 & = & \lambda\text{-clausura}(B,E) = \{B,D,E\} = q_2 \\
f(q_2,a) & = & \lambda\text{-clausura}(f(B,a)\cup f(D,a)\cup f(E,a)) \\
 & = & \lambda\text{-clausura}(C,E) = \{C,E\} = q_4 \\
f(q_2,b) & = & \lambda\text{-clausura}(f(B,b)\cup f(D,b)\cup f(E,b)) \\
 & = & \lambda\text{-clausura}(B,E) = \{B,D,E\} = q_2 \\
f(q_3,a) & = & \lambda\text{-clausura}(f(A,a)\cup f(B,a)\cup f(C,a)\cup f(D,a)\cup f(D,a)) \\
 & = & \lambda\text{-clausura}(B,C,A,E) = \{A,B,C,D,E\} = q_3 \\
f(q_3,b) & = & \lambda\text{-clausura}(f(A,b)\cup f(B,b)\cup f(C,b)\cup f(D,b)\cup f(D,b)) \\
 & = & \lambda\text{-clausura}(B,E) = \{B,D,E\} = q_2 \\
f(q_4,a) & = & \lambda\text{-clausura}(f(C,a)\cup f(E,a)) = \lambda\text{-clausura}(A,E) = \{A,C,E\} = q_5 \\
f(q_4,b) & = & \lambda\text{-clausura}(f(C,b)\cup f(E,b)) = \lambda\text{-clausura}(E) = \{E\} = q_6 \\
f(q_5,a) & = & \lambda\text{-clausura}(f(A,a)\cup f(C,a)\cup f(E,a)) \\
 & = & \lambda\text{-clausura}(B,A,E) = \{A,B,C,D,E\} = q_3 \\
f(q_5,b) & = & \lambda\text{-clausura}(f(A,b)\cup f(C,b)\cup f(E,b)) \\
 & = & \lambda\text{-clausura}(B,E) = \{B,D,E\} = q_2 \\
f(q_6,a) & = & \lambda\text{-clausura}(f(E,a)) = \lambda\text{-clausura}(E) = q_6 \\
f(q_6,b) & = & \lambda\text{-clausura}(f(E,b)) = \lambda\text{-clausura}(E) = q_6
\end{array}
$$

El AFD resultante aparece en la siguiente figura.

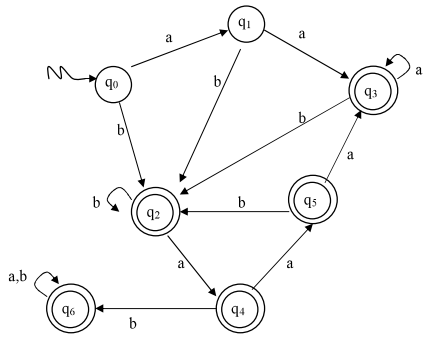


Figura 4.11: AFD equivalente al de la figura 4.10

Una vez obtenido el AFD, intentaremos minimizarlo.

Inicialmente, Partición $= \{G_1, G_2\}$ $\quad G_1 = \{q_2, q_3, q_4, q_5, q_6\}$ $\quad G_2 = \{q_0, q_1\}$

Analizaremos el comportamiento de todos los estados con a y b.

G_1		símbolo a	G_1		símbolo b
q_2	$\rightarrow$	$q_4 \in G_1$	q_2	$\rightarrow$	$q_2 \in G_1$
q_3	$\rightarrow$	$q_3 \in G_1$	q_3	$\rightarrow$	$q_2 \in G_1$
q_4	$\rightarrow$	$q_5 \in G_1$	q_4	$\rightarrow$	$q_6 \in G_1$
q_5	$\rightarrow$	$q_3 \in G_1$	q_5	$\rightarrow$	$q_2 \in G_1$
q_6	$\rightarrow$	$q_6 \in G_1$	q_6	$\rightarrow$	$q_6 \in G_1$

G_2		símbolo a	G_2		símbolo b
q_0	$\rightarrow$	$q_1 \in G_2$	q_0	$\rightarrow$	$q_2 \in G_1$
q_1	$\rightarrow$	$q_3 \in G_1$	q_1	$\rightarrow$	$q_2 \in G_1$

Todos los estados de G_1 son equivalentes, sin embargo, esto no ocurre con los de G_2, por lo que es necesario separarlos en dos elementos diferentes. La nueva y definitiva partición sería:

$\{G_1, G_2', G_3\}$ $\quad G_1 = \{q_2, q_3, q_4, q_5, q_6\}$ $\quad G_2' = \{q_0\}$ $\quad G_3 = \{q_1\}$.

A continuación se representa el nuevo autómata (figura 4.12) que es determinista, mínimo y equivalente al AFND del que partimos.